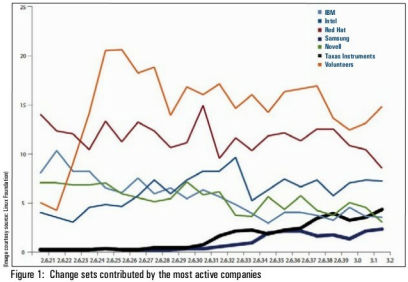

Figure 1: Change sets contributed by the most active companies

customised kernels that give high level support to their own system. Linux has already been ported to various kinds of hardware systems such as Raspberry Pi, ARM processors and thousands more. The design and size of Linux kernels running on these types of hardware varies greatly. Therefore, it becomes imperative to understand the Linux kernel and the process of configuring, modifying and compiling it.

The pace of Linux kernel releases

Linux kernel development proceeds under a loose, time-based release model, with a new major kernel release occurring every 2-3 months (from 2005 onwards). Linux version numbers follow a long standing tradition. Each version has three numbers, i.e., X.Y.Z. The X is only incremented when a really significant change happens, one that makes software written for one version no longer operate correctly on the other. This happened last in 2011 on the 20th birthday of Linux. The Y tells you which development 'series' you are in. A stable kernel will always have an even number in this position, while a development kernel will always have an odd number. The Z specifies which exact version of the kernel you have, and it is incremented on every release.

Let us look at some contribution statistics related to kernel development. The number of developers who are actively contributing to a given version of the kernel has steadily grown. Approximately 75 per cent of individual contributors are professional software developers who are paid to work on the kernel. Volunteer developers who are known to be contributing without compensation still represent a larger segment of contributors than developers from any given company. The top few companies that contributed in 2011 were Red Hat, Intel, IBM, and Novell. Interestingly, Google contributed less than Nokia, and Microsoft was the 17th most prolific contributor. The trend chart in Figure 1 depicts the percentage of change sets contributed by the most active companies since the 2.6.20 release in 2007.

Linux kernel development is fun and this is depicted in Figure 2.

From supercomputers to smartphones, the Linux kernel is a versatile and widely-used piece of technology. So let's have a peek into the process of compiling a Linux kernel.

Compiling a custom kernel

We will use the instructions given below to compile the current stable release of Linux kernel 3.10, which can be used in any Linux distribution.

Step 1

To set up the kernel development environment, install the *libncurses* library:

```
$ sudo apt-get install ncurses-bin libncurses5 // for ubuntu like systems
// use yum for fedora like systems
```

Step 2

Download the kernel as follows:

```
$ cd /usr/src/
$ sudo wget https://www.kernel.org/pub/linux/kernel/v3.x/linux-3.10.10.tar.xz
```

Step 3

The next step is to extract the kernel source file for configuring and compilation:

```
$ tar -xvJf linux-3.10.10.tar.xz
```

Step 4

Please note that the current Linux kernel contains more than 15 million lines of code (LoC). The kernel contains nearly 3000 options to configure. In 21 years, the kernel code has been expanded to support thousands of devices and services. A majority of the GNU/Linux distros provide the default kernel configuration to support most of the hardware available today. One can either use this default configuration and modify the necessary options, or use the default configuration that came with the kernel. In either case, be sure of the impact of modifying a particular option. Invalid configuration will result in the kernel going into panic mode, which will prevent the system from booting. However, you can always use the previously installed kernel to boot the system.

```
$ cd linux-3.10.10
$ sudo make menuconfig
```